DESAFIO 02

A Casa Sempre Ganha

O Jorjão quer cinco máquinas de roleta e só uma paga.
Quem monta a banca é você.

AMAZON EC2 · APPLICATION LB · AUTO SCALING · USER DATA · AMAZON S3

01 Contexto

Todo churrasco de família tem um. O seu se chama **Jorjão Fortuna**: primo de segundo grau, corrente dourada, fone no queixo, e a frase pronta: "fiquei rico com aposta". O detalhe que ele conta baixinho, depois do terceiro espeto, é **como**: o Jorjão não aposta. O Jorjão é **a banca**.

Agora ele vai lançar a **GiroBet**, com a lendária **Roleta Giratória Premiada**, e no último churrasco ele encostou em você, o parente que "mexe com computador" e que ele descobriu fazer parte da **Liga de Computação em Nuvem da PUCRS**.

Dias depois, chegou o áudio transcrito:

De: Jorjão Fortuna 🐊💸
Assunto: soc sociedade GiroBet 🎰🚀

Primão! Contratei uns guri pra fazer o site e a rôleta, ficou MONSTRO. Só que os guri não fazem essa parte de "nuvem" aí que tu manja.

Eu preciso de duas coisas, presta atenção:

Primeira: o site não pode cair NUNCA. Vô divulgar no jogo do Grêmio, vai entrar uma cacetada de gente girando junto. Se cair uma máquina, tem que ter outra no lugar, automático, sem eu fazer nada.

Segunda (aumenta o volume não): eu quero **cinco máquinas de roleta**, e **só uma paga prêmio**. As outras quatro são pra... dar volume, digamos assim. O apostador tem que ter a chance dele, uma em cinco, justíssimo, mas quem escolhe a máquina não é ele, entendeu?

Confio no teu taco, primão. Faz essa mágica que a gente conversa sobre a tua porcentagem. 🤝

Você leu duas vezes. Respirou. E percebeu duas coisas: primeiro, que o Jorjão descreveu, sem saber o nome de nada, um **load balancer** na frente de um **auto scaling group**. Segundo, que essa é a chance perfeita de aprender os dois, entregar o "cassino" fictício do primo e ainda mostrar para ele, com estatística, por que casa de aposta não é sorte.

O jogo do Grêmio é no domingo que vem. A banca precisa estar montada.

02 O que temos?

```
desafio02-a-casa-ganha/
├── backend/          → a API da roleta (a IMAGEM já vem pronta no GHCR)
├── frontend/dist/    → o site, JÁ BUILDADO (config do endereço em runtime)
└── estadio.mjs       → o teste de carga: o "estádio inteiro" do Jorjão
```

O repositório: https://github.com/Liga-de-Computacao-em-Nuvem-PUCRS/desafio02-a-casa-ganha

Duas novidades em relação ao Desafio 01:

- **Ninguém builda nada.** A imagem do backend está publicada no GHCR (`ghcr.io/liga-de-computacao-em-nuvem-pucrs/desafio-02-bet-backend`); suas instâncias só fazem `docker pull`. O frontend já vem em `dist/`.
- **A configuração virou runtime.** O endereço da API se edita no `dist/config.js`, sem rebuild.

O comportamento da roleta é deliberadamente simples: uma máquina com `PREMIADA=true` **sempre** paga; sem a variável, **nunca** paga. Não há sorteio no código. A probabilidade de 1 em 5 que o Jorjão pediu vai nascer inteira da **sua infraestrutura**.

O que **não** veio: o script de **user-data**, o diagrama, a estimativa de custos e qualquer configuração de nuvem.

03 Sua missão

Montar a banca do Jorjão: um Application Load Balancer distribuindo giros entre 5 instâncias idênticas criadas por um Auto Scaling Group, e provar, girando, que a probabilidade é 1 em 5.

01 Desenhar a rede: dois Security Groups

Um para o ALB (recebe o público na 80) e um para as máquinas, que aceita a porta 3000 **somente do Security Group do ALB**, e o SSH somente do seu IP. Critério de aceite: `curl` no IP público de uma instância, porta 3000, **falha**; o mesmo pedido, pelo DNS do ALB, **funciona**. Você precisa saber justificar cada regra, inclusive por que uma regra pode apontar para outro Security Group em vez de um IP.

SECURITY GROUPS

02 Escrever o user-data de um Launch Template

A máquina precisa nascer pronta: Docker instalado e o container da roleta no ar, **sem você entrar via SSH depois do boot**. É o seu primeiro script de bootstrap imutável: o mesmo texto vai criar as 5 máquinas, e qualquer correção se faz numa nova versão do template, nunca na máquina. Critério: uma instância recém-criada responde `/health` sozinha.

LAUNCH TEMPLATE · USER DATA

03 Criar o Target Group com health check no `/health`

Porta 3000, protocolo HTTP. Critério: os 5 targets `healthy`. Saiba explicar o que acontece com uma máquina que o health check reprova.

TARGET GROUP · HEALTH CHECK

04 Colocar o Application Load Balancer na frente

Público, HTTP na 80, em **pelo menos duas Availability Zones**. E descobrir no caminho por que o ALB se recusa a existir em uma só.

APPLICATION LOAD BALANCER

05 Criar o Auto Scaling Group

Desired, mínimo e máximo em **5**, usando o Launch Template. Critério de aceite: **termine uma instância na mão** e assista o ASG repô-la sem a sua ajuda. É o "não pode cair nunca" do Jorjão, funcionando.

AUTO SCALING GROUP

06 Marcar a máquina premiada

Entre via SSH em **uma** das cinco e recrie o container com `PREMIADA=true`. Critério de aceite: girando pelo site (use o modo turbo) ou soltando o `estadio.mjs` contra o DNS do ALB, a taxa de vitória fica em torno de **20%**, cada máquina recebe ~1/5 dos giros e a coroa aparece numa máquina só.

SSH · DOCKER

07 Publicar o site no S3

Bucket com hosting estático, `dist/config.js` editado com o DNS do ALB. Critério: o site funciona no endereço do bucket, para qualquer pessoa.

AMAZON S3 · STATIC HOSTING

08 Entregar o diagrama e a estimativa de custos para o Jorjão

O diagrama mostra o caminho de um giro, do navegador à máquina premiada, o que é público e o que não é. A estimativa traz o custo por serviço e as premissas. Este é o primeiro desafio que **sai do Free Tier**, então inclua o custo de uma tarde de laboratório e o custo de um mês esquecido ligado. E feche com a seção que o Jorjão pediu sem pedir: **"por que a casa sempre ganha"**, explicando com a matemática do próprio sistema (1 prêmio a cada ~5 giros, prêmio menor que 5 apostas) por que o lucro da banca não depende de sorte. Escrito para quem não sabe o que é um Security Group. O Jorjão não sabe nem o que é a nuvem, ele só sabe cobrar.

> ***Guarde o `.pem` como o cordão de ouro do Jorjão.** Baixa uma vez, `chmod 400`, fora de pasta versionada, e não empresta para ninguém.

ARQUITETURA · PRICING CALCULATOR

04 O combinado

- Região: **us-east-1**, do começo ao fim.
- **AWS Budget de US$ 5** configurado antes da primeira instância.
- Ao terminar, a limpeza segue a ordem certa. Neste desafio a ordem importa de verdade: quem terminar as instâncias antes de zerar o ASG vai vê-las renascer. Comece pela causa, não pelo sintoma.

05 Desafio aposta alta

Domingo, 22h. O site aguentou o jogo do Grêmio inteiro. Aí toca o telefone: é o Jorjão, em pânico.

"Primão, SUMIU O PRÊMIO! Um apostador ganhou às 21h30, eu vi no painel! Daí eu fui mexer lá naquele site da Amazon que tu me mostrou, cliquei sem querer em *Terminate* numa máquina... o site continuou no ar, ó, nota dez, NÃO CAIU... mas agora ninguém ganha MAIS NADA. Cadê minha máquina premiada, primão??"

09 Explicar (e demonstrar) o que aconteceu com a máquina premiada

Reproduza o desastre do Jorjão: termine a instância premiada e observe o ASG. Documente o que acontece com o site, o que acontece com o prêmio, e **por quê**. A resposta tem tudo a ver com onde o `PREMIADA=true` morava.

SELF-HEALING

10 Quebrar o 1-em-5 sem tocar em máquina nenhuma

Ative **sticky sessions** no Target Group e gire de novo, bastante. Documente o que acontece com a taxa de vitória de **um mesmo** navegador e explique por que essa configuração, que existe por bons motivos, destrói a premissa estatística da banca.

STICKY SESSIONS

Você vira o dono da banca quando

- Consegue explicar por que o *self-healing* do ASG recuperou o site mas **não** o prêmio, e onde esse estado deveria morar para sobreviver.
- A demonstração das sticky sessions mostra a taxa de vitória indo para **0% ou 100%**, e você sabe dizer qual dos dois e por quê, antes de testar.
- O diagrama final mostra o caminho do giro **com** o Auto Scaling Group repondo uma máquina.

> **Dica do crupiê:** o painel do site precisa mostrar o crachá verdadeiro de cada máquina, o `i-0abc...` de verdade. Se todas se apresentarem com um nome estranho de container, a resposta do serviço de metadados está morrendo um salto antes de chegar. Procure por **metadata response hop limit** e pense em quantos saltos existem entre o serviço de metadados e um processo dentro de um container.